

Scope delle variabili

Scope delle variabili

Dove le variabili esistono e dove sono accessibili nel codice.

Cos'è lo scope?

Immagina di avere una variabile `x` dentro una funzione. Quella `x` esiste solo dentro quella funzione. Fuori non la puoi usare, come se non esistesse.

Questo concetto si chiama **scope** (o "ambito"): la zona del programma in cui una variabile è accessibile.

Variabili locali — dentro la funzione

Una variabile creata dentro una funzione è **locale**: vive solo per la durata di quella funzione e poi scompare.

```
def mia_funzione():  
    x = 10    # variabile locale: esiste solo dentro questa funzione  
    print(x)  
  
mia_funzione() # 10  
# print(x)     # NameError: 'x' non esiste qui fuori!
```

Questo è un comportamento voluto, non un difetto. Significa che le funzioni sono **indipendenti**: le variabili dentro una funzione non possono interferire con quelle fuori.

Variabili globali — fuori dalle funzioni

Una variabile definita fuori da qualsiasi funzione è **globale**: è visibile da tutta la parte principale del programma.

```
nome = "Alice" # variabile globale

def saluta():
    print(f"Ciao, {nome}!") # può leggere la variabile globale

saluta() # Ciao, Alice!
```

Puoi leggere le variabili globali, ma non modificarle (di solito)

Una funzione può **leggere** una variabile globale, ma se prova a modificarla, Python crea una nuova variabile locale con lo stesso nome invece di cambiare quella globale:

```
contatore = 0

def incrementa():
    contatore = 1 # ATTENZIONE: crea una variabile locale, non modifica
    quella globale!
    print(f"Dentro la funzione: {contatore}")

incrementa()
print(f"Fuori dalla funzione: {contatore}") # ancora 0, non è cambiato!
```

Se vuoi davvero modificare una variabile globale dall'interno di una funzione, devi usare la parola chiave `global` :

```
contatore = 0

def incrementa():
    global contatore # ora si riferisce alla variabile globale
    contatore += 1

incrementa()
incrementa()
print(contatore) # 2
```

...tip[Consiglio] Usare `global` è generalmente una cattiva abitudine. Rende il codice difficile da capire e da correggere. È meglio passare i valori come parametri e usare `return` per restituirli. ...

Variabili con lo stesso nome a livelli diversi

Se hai una variabile locale e una globale con lo stesso nome, dentro la funzione vince quella locale:

```
x = "globale"

def funzione():
    x = "locale"
    print(x)  # locale – la variabile locale ha la precedenza

funzione()
print(x)      # globale – fuori dalla funzione, x è ancora quella globale
```

Non vengono confuse: sono due variabili separate che condividono solo il nome.

Le funzioni predefinite di Python

Funzioni come `print`, `len`, `range`, `input` sono sempre disponibili ovunque nel codice. Python le cerca per ultime, ma le trova sempre.

Per questo non dovresti mai chiamare una tua variabile `print` o `list`: sovrascriveresti la funzione predefinita e poi non potresti più usarla!

```
# Da non fare mai!
# list = [1, 2, 3]  – ora non puoi più usare list() come funzione
# print = "ciao"   – ora print() non funziona più!
```